

Proyecto Refactorizado: Documentación Completa

1. Estructura del Proyecto

Observación: Todas las claves de acceso son 1234.
Usuario bd root sin clave.

El proyecto se basa en la gestión de usuarios (clientes, administradores) en una aplicación web desarrollada en PHP, siguiendo un enfoque orientado a objetos. El objetivo principal del proyecto es permitir la administración eficiente de usuarios, proporcionando diferentes niveles de acceso y funcionalidades específicas según el rol (usuario, editor o administrador). Los roles disponibles son usuario, editor y administrador. Se eliminó el rol de superadministrador.

2. Archivos Involucrados y Sus Roles

- ****BaseDeDatos.php****: Contiene la clase `BaseDeDatos` que maneja la conexión con la base de datos y las consultas comunes.
- ****Usuario.php****: Contiene la clase `Usuario` con todos los métodos relacionados con la gestión de los usuarios, incluyendo `crearUsuario()`, `editarUsuario()`, `eliminarUsuario()`, y `listarUsuariosConBusquedaOrden()`.
- ****Articulo.php****: Contiene la clase `Articulo` para la gestión de artículos, permitiendo al editor y al administrador crear, editar, eliminar y listar artículos.
- ****funciones.php****: Contiene funciones reutilizables para verificación de sesión, manejo de errores, y otros auxiliares.

3. Refactorizaciones Realizadas

1. ****Uso de Clases****: Todas las operaciones de usuarios ahora se manejan a través de la clase `Usuario`. Las funciones anteriores (como `crearUsuario()`, `editarUsuario()`, y `eliminarUsuario()`) fueron eliminadas de `funciones.php` y migradas a métodos dentro de la clase `Usuario`. Esto mejoró la cohesión del código, centralizando la lógica de gestión de usuarios y facilitando su mantenimiento.
2. ****Método `editarUsuario()`****: Se agregó la capacidad de editar el DNI del usuario, además de los datos ya existentes. También se validó que el DNI ingresado sea válido antes de realizar la actualización. Se actualizó la verificación de duplicados del DNI para que no considere el DNI del propio usuario que está siendo editado, evitando falsos positivos de duplicación.
3. ****Manejo de Eliminación de Cuenta****: Para la eliminación de la cuenta de un usuario que está actualmente logueado:
 - Se usó el método `eliminarUsuario(\$usuario\_id)` de la clase `Usuario`.
 - Se cierra la sesión del usuario (`session\_destroy()`) después de la eliminación y se redirige al usuario a la página principal (`index.php`).
 - Se añadió un mensaje de alerta que notifica al usuario de la eliminación exitosa antes de redirigir.
4. ****Validación de DNI****: Todos los métodos de creación y edición de usuarios fueron actualizados para incluir la validación del DNI según la normativa española.
5. ****Manejo de Artículos****: Se creó la clase `Articulo` para la gestión de artículos. Los editores y administradores pueden dar de alta, modificar, eliminar, buscar y ordenar artículos. Las operaciones

relacionadas con artículos también fueron migradas a esta clase para mantener una estructura clara y cohesiva.

4. Ejemplo de Refactorización en `editar\_usuario.php`

En el archivo `editar\_usuario.php`, se implementó la siguiente lógica:

- Se verifica si el usuario desea actualizar sus datos o eliminar la cuenta.
- Si se presiona el botón de eliminación, se llama al método `eliminarUsuario(\$usuario\_id)`, se destruye la sesión y se muestra un mensaje de confirmación antes de redirigir al usuario.

```
```php
if (isset($_POST['eliminar'])) {
 try {
 // Eliminar el usuario usando el método de la clase Usuario
 $usuario->eliminarUsuario($usuario_id);

 // Mostrar mensaje de éxito antes de destruir la sesión y redirigir
 echo '<script>
 alert("Cuenta eliminada exitosamente. Serás redirigido al inicio de sesión.");
 </script>';

 // Cerrar sesión después de eliminar la cuenta
 session_destroy();

 // Redirigir a la página principal o de inicio después de mostrar el mensaje
 echo '<script>
 window.location.href = "index.php";
 </script>';
 exit;
 } catch (Exception $e) {
 $error = "Error al eliminar la cuenta: " . $e->getMessage();
 }
}
```
```

5. Problemas Comunes y Soluciones

- ****Actualización de Datos No Reflejada en el Formulario****: Después de actualizar los datos del usuario, el formulario mostraba los datos antiguos. Se resolvió volviendo a obtener los datos actualizados tras la edición, utilizando el método `obtenerUsuarioPorId()` para recargar los datos del usuario después de que se completó la actualización. Esto garantiza que la información reflejada en el formulario siempre esté sincronizada con la base de datos.
- ****Eliminación del Usuario Logueado****: El proceso de eliminación no estaba funcionando porque la sesión seguía activa. Se resolvió agregando `session\_destroy()` y redirigiendo después de la eliminación.
- ****Verificación de DNI Existente al Editar****: Se agregó lógica al método `editarUsuario()` para excluir al propio usuario al verificar si el DNI ya está registrado en la base de datos. Esto permite que el usuario mantenga su DNI sin errores de duplicación durante la edición.

- **Gestión de Artículos**: Los editores y administradores pueden gestionar los artículos mediante la clase `Articulo`. Se implementaron validaciones para garantizar que el código del artículo sea único y siga el formato requerido (tres letras seguidas de hasta cinco números).

6. Resumen de la Clase `Usuario`

La clase `Usuario` contiene los siguientes métodos clave:

- **crearUsuario()**: Crea un nuevo usuario en la base de datos, con validación de DNI.
- **editarUsuario()**: Edita los datos de un usuario, incluyendo el DNI, y valida que sea correcto. Se evita la verificación de duplicados contra el mismo DNI del usuario que está siendo editado.
- **eliminarUsuario()**: Elimina un usuario de la base de datos, incluyendo el cierre de sesión si es el usuario logueado.
- **listarUsuariosConBusquedaOrden()**: Lista los usuarios, con capacidades de búsqueda y ordenación.

7. Resumen de la Clase `Articulo`

La clase `Articulo` contiene los siguientes métodos clave:

- **crearArticulo()**: Crea un nuevo artículo en la base de datos, con validación del código para evitar duplicados.
- **editarArticulo()**: Edita los datos de un artículo, incluyendo la imagen, si es necesario. Valida que el código del artículo no esté duplicado.
- **eliminarArticulo()**: Elimina un artículo de la base de datos.
- **listarArticulos()**: Lista los artículos, con capacidades de búsqueda, paginación y ordenación.
- **contarArticulos()**: Cuenta el total de artículos para la paginación.

8. Recomendaciones para Futuros Pasos

- **Evitar Carga de Información Repetitiva**: Optimización del uso de funciones y recursos para evitar latencias excesivas. Por ejemplo, evitar realizar múltiples consultas a la base de datos en bucles y, en su lugar, intentar agrupar las consultas siempre que sea posible. También se recomienda centralizar las funciones de verificación y validación en métodos reutilizables, reduciendo el código duplicado.
- **Implementar Tokens CSRF**: Como se tuvo problemas previos con la implementación de protección mediante tokens CSRF, proceder con cuidado y validar cada paso para una mayor seguridad.
- **Refactorizar Otros Módulos**: Continuar con la refactorización del código para utilizar métodos orientados a objetos en todo el proyecto.
- **Mejorar la Gestión de Roles**: Añadir más granularidad a los permisos de cada rol, para asegurar que solo los usuarios adecuados puedan realizar ciertas acciones específicas.

9. Registro de Usuario: Mensaje de Confirmación y Redirección

En la lógica del registro de usuario se añadió la funcionalidad para, después de un registro exitoso, mostrar un mensaje de confirmación al usuario y luego redirigirlo automáticamente a la página de inicio de sesión (`login.php`). Esta es una buena práctica desde el punto de vista de la experiencia del usuario, ya que les proporciona una guía clara sobre cuál es el siguiente paso a seguir, evitando confusiones sobre cómo proceder después de crear una cuenta.

Código sugerido para el final del bloque `if($_SERVER["REQUEST_METHOD"] == "POST")` en

`registro.php`:

```
``php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    try {
        // Recibir y sanitizar los valores del formulario
        $dni = trim(strtoupper($_POST['dni']));
        $nombre = trim(htmlspecialchars($_POST['nombre']));
        $correo = trim($_POST['correo']);
        $telefono = trim($_POST['telefono']);
        $direccion = trim(htmlspecialchars($_POST['direccion']));
        $localidad = trim(htmlspecialchars
```